

****BidWise Auction Platform: Complete Guide****

****Introduction****

BidWise is an online auction platform where:

- ****Sellers**** list items for auction
- ****Buyers**** bid on items
- ****Admin**** manages users and auctions

Built with:

- ****Frontend****: Next.js (user interface)
- ****Backend****: Node.js (server logic)
- ****Database****: MongoDB (data storage)
- ****Image Hosting****: Cloudinary

****1. Project Setup****

****Install Essentials****

1. ****[VS Code]**** ----- <https://code.visualstudio.com> ----- Code editor
2. ****[Node.js v20.18.1]**** — <https://nodejs.org> -----js runtime
3. ****[Git]**** ----- <https://git-scm.com/> -----Version control system

4. **pnpm** (run after Node.js installation): faster than npm.

Installs the package manager

In terminal: `npm install -g pnpm`

****Get the Code****

```In terminal:

# *Frontend*

`git clone https://github.com/eauctionproject2025/eAuction.git`

# *Backend*

`git clone https://github.com/eauctionproject2025/backend.git`

```

Note: Creates two folders: `eAuction` (website) and `backend` (server)

****Configure Environment****

****Frontend (`eAuction/.env`):****

```env

`NEXTAUTH_SECRET=5d163f1e3f36fca36f97f1bcf03571af75105621b3631d5d42bc392b7af34e70ddc06f61629f3715b1db324c83842357669741437fcf97e6dcfe5dfa39f82df6`

`NEXT_PUBLIC_FRONTEND_URL=http://localhost:3000`

`NEXT_PUBLIC_BACKEND_URL=http://localhost:5000/`

```

****Backend (`backend/.env`):****

```
```env
```

```
DB_USER=eauctionproject2025
```

```
DB_PASS=natWplxEYQPVBRII
```

```
JWT_SECRET=5d163f1e3f36fca36f97f1bcf03571af75105621b3631d5d42bc392b7af34e70ddc06f61629f3715b1db324c83842357669741437cf97e6dcfe5dfa39f82df6
```

```
NEXT_PUBLIC_FRONTEND_URL=http://localhost:3000
```

```
CLOUDINARY_CLOUD_NAME=di5ieqrx
```

```
CLOUDINARY_API_KEY=398565714277968
```

```
CLOUDINARY_API_SECRET=PY3dQhIru28i2hZVww80WKfLAiM
```

```
```
```

****Launch Application****

```
| **Frontend**      | **Backend** |
```

```
|-----|-----
```

```
| ```bash           | ```bash
```

```
| cd eAuction       | cd backend
```

```
| pnpm install       | pnpm install      # Installs dependencies
```

```
| pnpm run dev       | node index.js     # Starts servers
```

```
**Access:** http://localhost:3000
```

```
---
```

****2. Core Functionality Explained****

****User Authentication****

****Registration Flow:****

1. ****Frontend**** (`src/app/(auth)/register/page.jsx`):

Function: [handleRegister\(\)](#)

- Collects: Name, Email, Password, NID
- Validates: Email format, password strength
- Calls: POST `/api/auth/register`

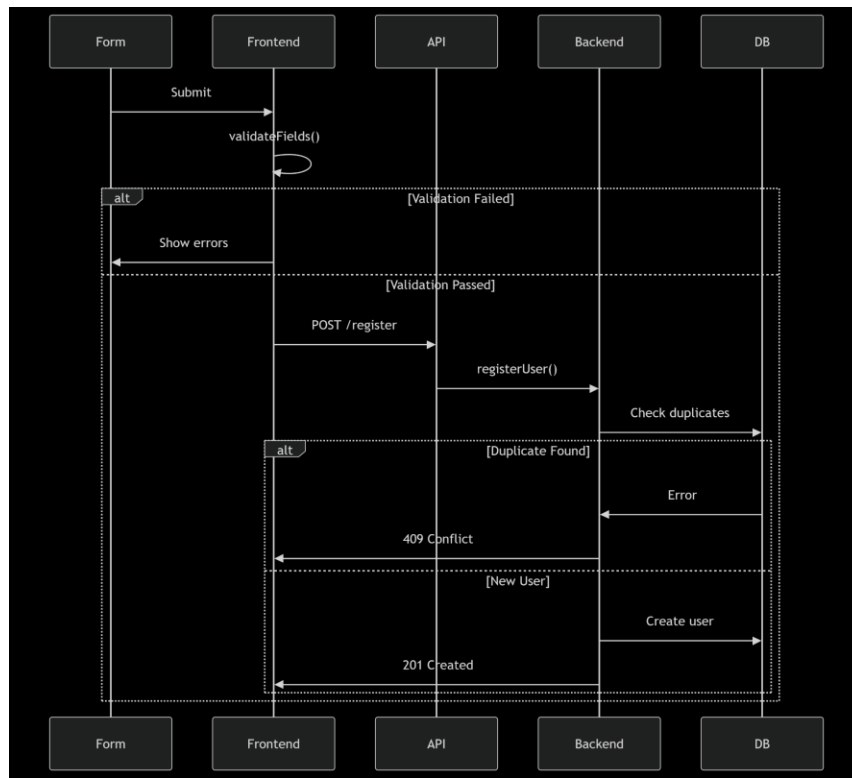
2. ****API Call****: POST `/api/auth/register`

3. ****Backend**** (`controllers/authController.js`):

Function: [registerUser\(\)](#)

- Checks: Email/NID uniqueness in database
- Secures: Password encryption
- Creates: New user profile
- Returns: Success/error message

4. ****Database****: Saves to MongoDB via `models/User.js` schema



****Login Flow:****

1. **Frontend** (`src/app/(auth)/login/page.jsx`):

Function: [handleLogin\(\)](#)

- Extracts form values (email, password, role)
- Uses NextAuth's signIn to verify credentials
- Redirects to Home page/admin dashboard

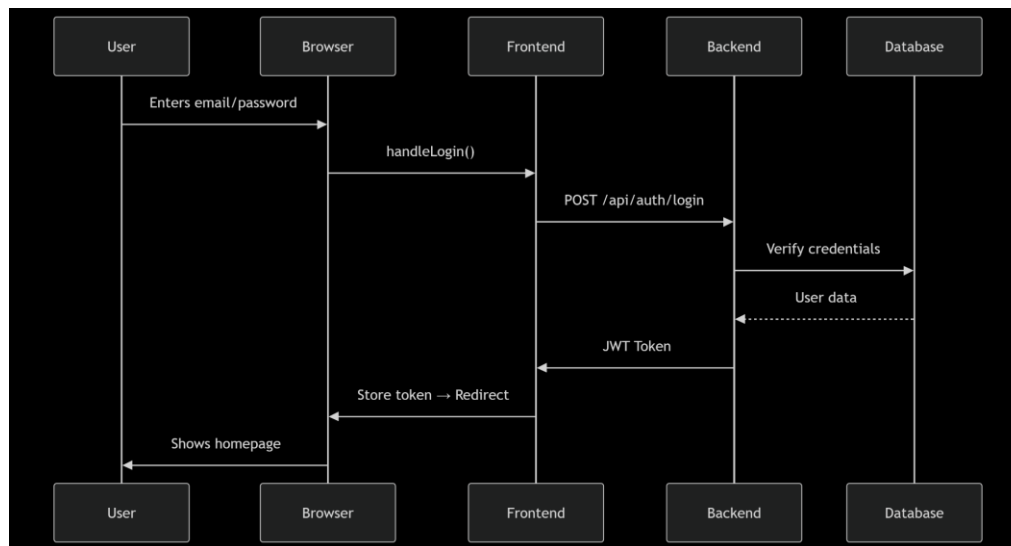
2. **API Call:** POST `/api/auth/login`

3. **Backend:**

Function: [loginUser\(\)](#)

- Verifies: Credentials match
- Generates: Secure session token
- Returns: User data + permissions

4. **Session:** Creates JWT token for security



****Sign Out:****

File: components/navbar.jsx

Function: `signOut()`

- Clears: All session data
- Redirects: To login page

—

**Navbar**

File: ("components/Navbar.jsx")

****Scroll Behavior****

- Hides navbar on scroll down (via `useEffect` scroll listener)
- Shows on scroll up

****User Management****

- `handleSignOut()`: Clears session and signs out
- Displays user menu based on auth state

****Navigation****

- Main menu links (All Auction, Create Auction, etc.)
- Responsive mobile menu toggle

****Auth Integration****

- Shows profile pic/admin links based on session.user.role

**Greet**

File: ("components/Greet.jsx")

Welcome System:

- Auto-shows welcome message for logged-in users
- Auto-hides after 3 seconds
- Uses CSS animations for smooth transitions

**Homepage**

Path: ("app/page.js")

Core Functions:

****Data Fetching****

- `fetchAuctions()`: GET /api/auctions → Loads auctions
- `fetchCategories()`: GET /api/categories → Loads top 5 categories

****User Greeting****

- Shows welcome message (Greet component) on first login
- Uses sessionStorage to track first visit

****Key Components****

- CarouselSlide: Displays promotional banners
- Auction: Renders `Auction.jsx` component
- CategoryCard: Shows category previews

****Footer****

File: ("component/footer.jsx")

Core functionality:

- Import [footerBox](#) component for quick link
- Shows social media links

****LayoutShell****

File: ("components/LayoutShell.jsx")

Layout Logic:

Conditionally renders:

- Navbar (render everywhere except dashboard)
- Home page for all
- Footer (render everywhere except dashboard)

Handles path-based styling (dashboard vs public pages)

Routing Awareness:

- Uses `usePathname()` to check for /admin routes

****Root Layout****

File: ("app/layout.js")

- Wraps entire app with:

- `SessionProvider` (authentication context)
- [LayoutShell](#) (consistent page structure)

- Applies global fonts (Geist) and styles

- Sets default metadata

****Auction Management****

****Create Auction:****

1. **Frontend** (`components/auctionForm.jsx`):

Function:

handleSubmit()

- Gathers: Title, Description, Images
- Validates: Required fields

handleDelete()

- Confirms deletion
- Sends DELETE to `/api/auctions/:id`
- Triggers onDelete callback if successful

2. **API Call**: POST `/api/auctions``

3. **Backend** (controllers/auctionController.js):

Function:

a. getAllAuctions()

- Get's all auction
- Sorts by newest first

b. createAuction()

- Processes: Multiple images via Multer middleware
- Uploads: To Cloudinary storage
- Saves: Auction data to MongoDB
- Returns: New auction ID

c. getAuctionById()

- Auto-determines winner if auction ended
- Returns populated bidder/seller info

d. deleteAuction()

Security Checks:

- Owner or admin only
- Cleanup:
- Deletes Cloudinary assets
- Removes DB record

e. getAuctionsByUserId()

Role-aware:

- Sellers: Items they listed
- Buyers: Items they bid on

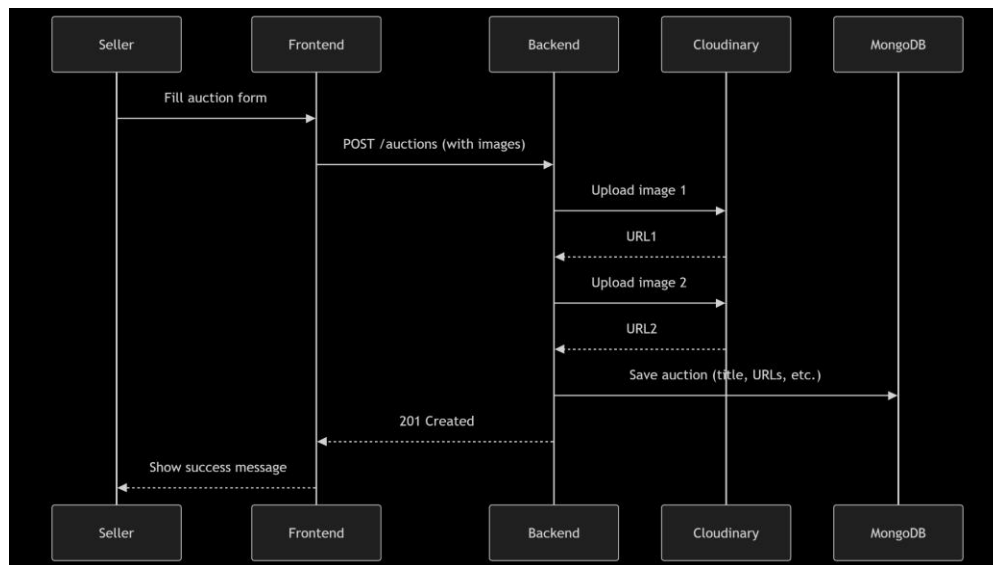
4. ****Special Features****:

- Owner/admin detection (isOwner, admin)
- Countdown timer integration
- Protected delete action (requires auth)

5. ****Error Handling Matrix****:

| <i>Scenario</i> | <i>Status Code</i> | <i>Response</i> |
|------------------------|---------------------------|------------------------------------|
| No images uploaded | 400 | {message: "No images uploaded"} |
| Invalid bid amount | 400 | {message: "Bid must be higher..."} |
| Unauthorized delete | 403 | {message: "Not authorized..."} |
| Auction not found | 404 | {message: "Auction not found"} |
| Cloudinary error | 500 | {message: "Image upload failed"} |

6. ****Diagram****



****Countdown Timer****

Purpose: Tracks and displays auction time status

Timer States

- ☐ Pre-Auction: Shows countdown until bidding starts
- ☐ Active Auction: Live countdown of remaining time
- ☐ Ended: Clear "Auction ended" notification

Key Features

- Auto-updates every second
- Color-coded for instant status recognition
- Simple time format (hours, minutes, seconds)

****Place Bid:****

1. ****Frontend**** (`app/items/[id]/page.jsx`):

Function: [handleBidSubmit\(\)](#)

- Validates: Bid amount
- Calls: POST /api/bids

2. ****API Call****: POST ` /api/bids`

3. ****Backend Validation****:

Function:

`placeBid()`

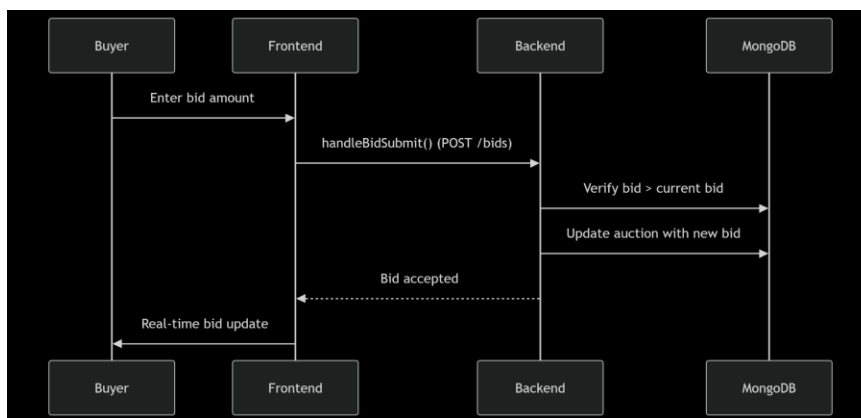
- Verifies: Bid > current bid
- Updates: Auction document
- Real-time Update

GET /bids - Fetch User's Bids

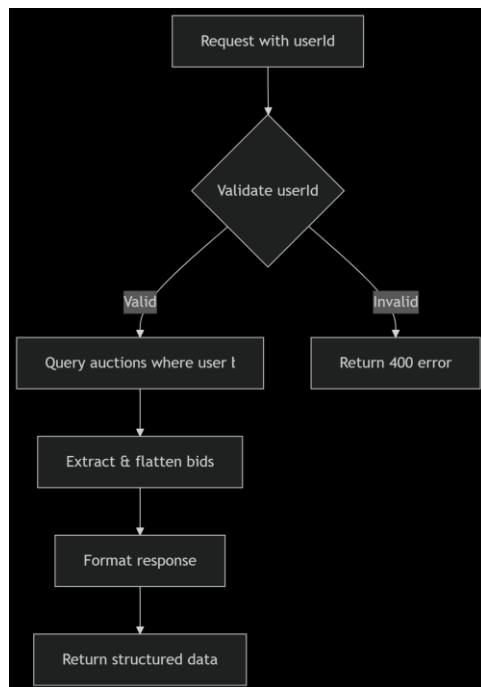
- Retrieves all bids placed by a specific user
- Returns enriched bid data with auction details

4. ****Diagram****

Bidding flow



Buyer's bid history



****Profile****

Profile Header

- Displays user image, name, role
- Shows edit controls for owners

Auction History

- Seller: Shows items listed
- Buyer: Shows bidding history

Data Fetching

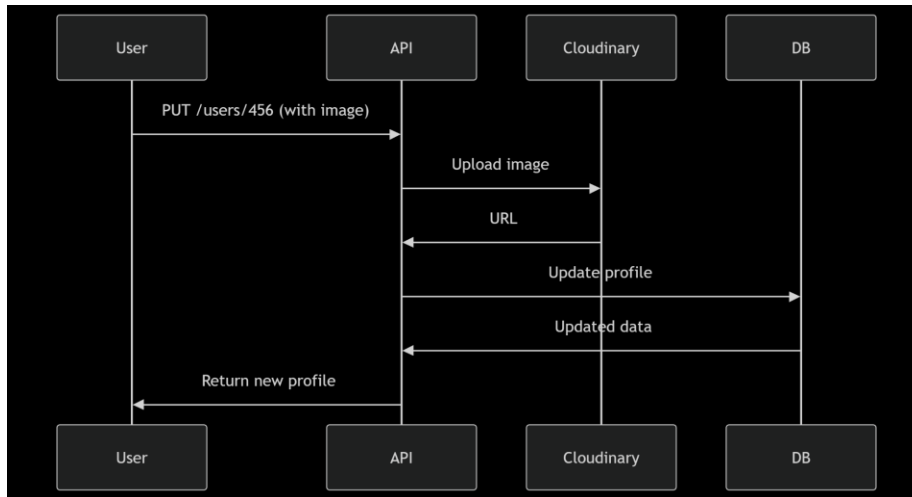
fetchProfile() (in [useEffect](#)):

- GET /api/users/\${userId} → Fetches user profile
- GET /api/auctions/user/\${userId} → Gets user's auction history
- Checks if viewer is profile owner (isOwner)

Profile Editing

Function: `handleUpdate()`

- POST /api/uploads → Uploads new profile image
- PUT /api/users/\${userId} → Updates profile data
- Handles: image, description, address updates



****Category****

1. ****Frontend**** (`app/category/page.jsx`):

Functions:

`fetchCategories()`

- GET /api/categories
- Displays loading state
- Passes data to CategoryCard components

1. ****Backend**** (`/routes/categoryRoute.js`):

- Router/get : List all categories

****Category/Auctions****

Function: `fetchAuctions()`

- GET /api/auctions

- Filters by category name (from URL)
- Handles loading states

****Admin Operations****

****Admin Login ****

File: app/admin/page.jsx

Function : [handleLogin\(\)](#)

- Validates admin credentials via NextAuth
- Redirects to dashboard on success
- Blocks non-admin roles with error messages

Dashboard *

File: ("admin/dashboard/page.jsx")

1. Data Fetching: [fetchAllData\(\)](#)

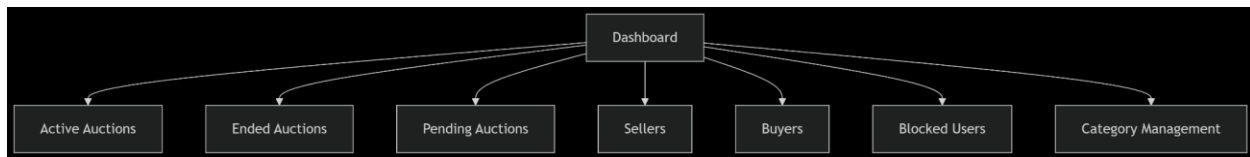
Uses [Promise.all](#) to simultaneously fetch:

- Auctions: [getActiveAuctions\(\)](#), [getEndedAuctions\(\)](#), [getPendingAuctions\(\)](#)
- Users: [getSellers\(\)](#), [getBuyers\(\)](#), [getBlockedUsers\(\)](#)
- Updates state with [setData\(\)](#)
- User [Summary Card](#) Component for rendering
- Handles errors during loading

2. Rendering Logic:

- Displays loading state while fetching data
- Calculates total auctions: active + ended + pending
- Shows top 5 active auctions via `activeAuctions.slice(0, 5)`

3. Diagram:



****Auction Management****

File: ("admin/dashboard/FetchAuction.js")

1. All Auctions:

`getAuctions()`

- Fetches all the Auctions details

2. Active Auctions:

`getActiveAuctions()`

- Fetches all the active Auction
- Render `AuctionList.jsx` to show them

3. Pending Auctions:

`getPendingAuctions()`

- Fetches all the pending Auction
- Render `AuctionList.jsx` to show them

4. Ended Auctions:

`getEndedAuctions()`

- Fetches all the ended Auction
- Render `AuctionList.jsx` to show them

****User Management****

File: ("admin/dashboard/FetchUser.js")

1. All Users:

`geUsers()`

- Fetches all the Users details

2. Sellers:

[getSellers\(\)](#)

- Fetches all the sellers details
- Render [UserList.jsx](#) to show their data

3. Buyers:

[getBuyers\(\)](#)

- Fetches all the buyers details
- Render [UserList.jsx](#) to show their data

4. Blocked User:

[getBlockedUsers\(\)](#)

- Fetches all the blocked user
- Render [UserList.jsx](#) to show their data.js`):

****Category Management****

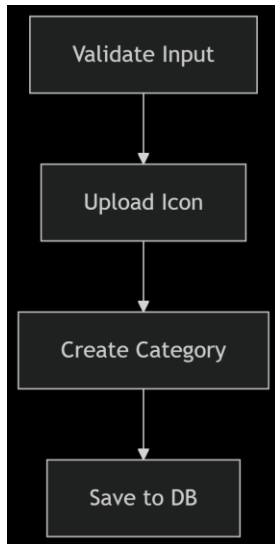
****Endpoint Structure****

| - | <u>Method</u> | <u>Path</u> | <u>Protection</u> | <u>Purpose</u> |
|---|----------------------|--------------------|--------------------------|-----------------------|
| - | POST | / | Protected | Create new category |
| - | GET | / | Public List all | categories |
| - | DELETE | /:id | Protected | Delete category |

****Core Functions****

1. [createCatergory\(\)](#)

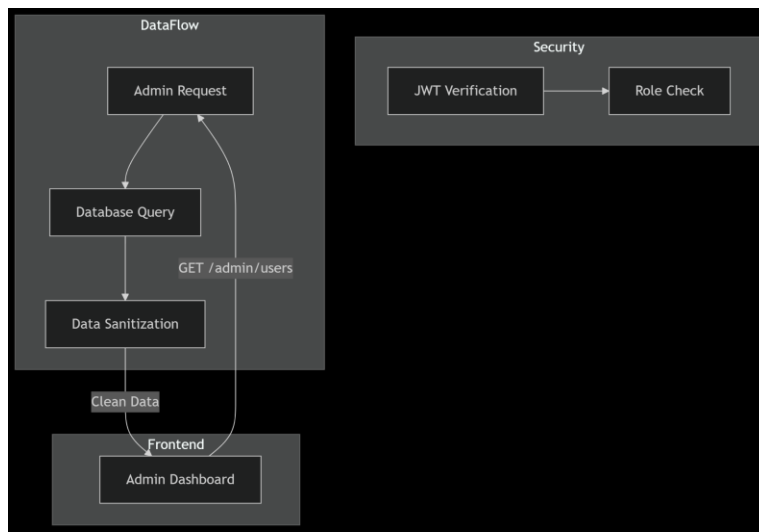
- Validate category Input
- Upload category Icon
- Save in DB with icon link form cloudinery



2. `deleteCategory()`

- Deletes DB record
- Removes Clouinary asset

****Admin data flow diagram****



****3. Component & Function Mapping****

****Frontend Architecture****

| **Component** | **Key Function** | **Connection** |
|----------------------------------|---|---------------------------|
| ----- ----- ----- | | |
| app/(auth)/register/page.jsx | handleRegister() | POST /api/auth/register |
| app/(auth)/login/page.jsx | handleLogin() | POST /api/auth/login |
| components/Navbar.jsx | handleSignOut() | Clears session |
| components/Greet.jsx | auto-show welcome | Session based |
| components/LayoutShell.jsx | layout wrapper | shows navbar/footer |
| components/Footer.jsx | shows footer | |
| components/CarouselSlide.jsx | slides | homepage |
| components/CategoryCard.jsx | render icon + name | homepage/category |
| components/AuctionList.jsx | fetchAuctions() | GET /api/auctions |
| components/AuctionForm.jsx | handleSubmit() → create
 handleDelete() → delete | POST/DELETE /api/auctions |
| components/CountdownTimer.jsx | calculateTimeLeft() | |
| components/CategoryManager.jsx | handleAddCategory()
 handleDeleteCategory() | admin |
| app/items/[id]/page.jsx | handleBidSubmit() | POST /api/bids |
| app/profile/[accountId]/page.jsx | fetchProfile()
 fetchUserAuctions()
 handleUpdate() | GET/PUT /api/users/:id |
| app/admin/page.jsx | handleLogin() | admin login |
| app/admin/dashboard/page.jsx | fetchAllData() | dashboard summary |
| admin/dashboard/FetchAuction.js | getAuctions(), getActiveAuctions(),
 getPendingAuctions(), getEndedAuctions() | |

| admin/dashboard/FetchUser.js | getUsers(), getSellers(), getBuyers(),
getBlockedUsers() ||

| admin/dashboard/UserList.jsx | render user list ||

| admin/dashboard/AuctionList.jsx | render auction list ||

Backend Architecture

| ****File**** | ****Key Function**** | ****HTTP Route****

|-----|-----|-----

| controllers/authController.js | registerUser()
loginUser() | /register,
/login |

| controllers/auctionController.js |
createAuction()
getAllAuctions()
getAuctionById()
deleteAuction
()
placeBid()
getAuctionsByUserId() | auctions & bids |

| controllers/adminController.js |
blockUser()
createCategory()
deleteCategory() | admin only |

| middleware/upload.js | upload.array() | images |

| models/User.js | schema ||

| models/Auction.js | schema ||

| models/Category.js | schema ||

| routes/authRoutes.js | POST /register//login ||

| routes/auctionRoutes.js | POST/GET/DELETE /auctions
POST /bids ||

| routes/adminRoutes.js | PATCH /users/:id/block
POST/DELETE
/categories ||

| index.js | Starts server | port 5000 |

4. Error Handling

Frontend Example:

javascript

```
if (error.message.includes("Email/NID")) {  
    alert("Email or NID already registered");  
}  
  
if (amount <= highestBid) {  
    return res.status(400).json({ message: "Bid must be higher than  
current bid" });  
}
```

Backend Framework:

javascript

```
app.use((err, req, res, next) => {  
    console.error(err.stack);  
    res.status(500).json({ error: "Internal server error" });  
});  
...
```

****Common Errors:****

- `400 Bad Request`: Invalid input (e.g., wrong email format)
- `401 Unauthorized`: Expired/missing login token
- `404 Not Found`: Auction/user doesn't exist
- `500 Server Error`: Database/cloud service failure

5. Third-Party Integration

****Cloudinary + Multer Workflow:****

1. Multer (`upload.js`) receives images
2. Converts to buffer format
3. Cloudinary SDK uploads with API keys
4. Returns CDN URLs:

```
```javascript
{
 "public_id": "auctions/item123",
 "secure_url": "https://res.cloudinary.com/.../image.jpg"
}
```
```

****MongoDB Atlas Connection:****

- `db.js` establishes connection:

```
```javascript
mongoose.connect(
 `mongodb+srv://${process.env.DB_USER}:${process.env.DB_PASS}@cluster0.mongodb.net/auctionDB`
);
```
```

6. Full System Map

Frontend Structure

...

src/

├─ app/

| └─ (auth)/

| | └─ login/ → handleLogin()

| | └─ register/ → handleRegister() [Validates inputs]

| └─ admin/

| | └─ dashboard/ → handleBlockUser() [Admin controls]

| └─ createAuction/

| | └─ page.jsx → handleSubmit() [Auction creation]

| └─ items/[id]/

| | └─ page.jsx → handleBidSubmit() [Bidding]

| └─ profile/[accountId]/

| └─ page.jsx → fetchUserData() [Profiles]

└─ components/

| └─ AuctionList.jsx → fetchAuctions() [Real-time updates]

| └─ auctionForm.jsx → validateImages() [File checks]

| └─ CategoryManager.jsx → handleAddCategory() [Admin feature]

...

****Backend Structure****

...

backend/

|— controllers/

| |— authController.js → registerUser() [Unique email/NID check]

| |— auctionController.js → createAuction() [With image processing]

| |— adminController.js → blockUser() [Toggle status]

|— middleware/

| |— upload.js → upload.array() [Multer+Cloudinary]

|— models/

| |— User.js → {name, email, role, nid, ...}

| |— Auction.js → {title, bids, images, ...}

| |— Category.js → {name, createdAt}

|— routes/

| |— authRoutes.js → POST /login, /register

| |— auctionRoutes.js → POST /auctions, GET /auctions

| |— adminRoutes.js → PATCH /users/:id/block

|— index.js → Starts server (Port 5000)

...

7. Support & Resources

- **Frontend Code**: <https://github.com/eauctionproject2025/eAuction>

- **Backend Code**: <https://github.com/eauctionproject2025/backend>

NoTe;

1.front end and backend how many pages.

Before login

(login , registration,homepage,category page,product page,products details page,profile page)

-7

After login

(profile page,add auction page) -2

Bacdkend end page:

(admin login page,admin dashboard page-(active ,pending,ended,seller list,buyer list,block list),category management- 3

Github -source code, (all update)